

SPOTIFY PROJECT

Final Technical Report

COURSE	STACK	DELIVERY
Web Programming - S2026	React 19 + Django 5.2	Docker + PWA + Realtime

This report maps the delivered repository to the assignment, records team ownership, architecture, access rules, testing evidence, AI-assisted work, and known environment-dependent validation.

1. Team ownership by phase

Contributor / student ID	Phase 1	Phase 2	Phase 3 / final
Mahyar Rezaei Pourkhalili 402106037	Shared UI integration and backend contract planning.	Operational backend: artist verification, support, notifications, subscriptions and payment flow.	Payout/audit APIs, PWA, Docker, group listening, entitlement hardening, integration and final report.
Ali Mirzaei Feyzabadi 402106672	Music browsing and player-facing UI components.	Catalog domain: songs, albums, playlists, artist works and playback integration.	Upload UX, cover/audio integration, statistics presentation and catalog regression review.
Mohammad Parsa Hassanzadeh 402106423	Application foundation, navigation, authentication and shared React structure.	Profiles, preferences, following, subscription UI and shared frontend tests.	Cross-feature testing, responsive polish and delivery review.

Working conventions

- Feature-oriented React modules and Django apps are split by business domain; pages delegate HTTP work to typed services.
- REST payloads use snake_case at the API boundary and map explicitly to camelCase UI models.
- JWT authentication, object ownership, role permissions, verified-artist checks and subscription entitlements are enforced server-side.
- Model changes include migrations; mutations use serializers/domain services; regressions receive focused API or service tests.
- Secrets and deployment values come from environment variables. PostgreSQL, Redis, media and WebSocket routing are defined in Compose.
- Git work is committed in reviewable feature slices; generated outputs, dependency folders, caches and local secrets remain excluded.

Ownership descriptions reflect repository evidence and the corrected student identity list supplied by the team.

2. Architecture and request flow

Layer	Technology	Responsibilities
Client	React 19, TypeScript, Vite	Responsive screens, player state, forms, PWA shell, API and WebSocket clients.
API	Django 5.2, DRF	Authentication, validation, catalog CRUD, limits, roles, reports and payment orchestration.
Realtime	Django Channels, Redis	Temporary listening rooms, membership, synchronized song/play/pause/progress events.
Data	PostgreSQL	Users, preferences, catalog, streams, tickets, notifications, transactions, payouts and rooms.
Edge	Nginx	Static frontend delivery and reverse proxying for HTTP, media and WebSocket upgrades.

Request flow

Browser traffic enters Nginx. Static navigation is handled by React; /api reaches Django/Daphne and /ws upgrades to Channels. Django authenticates JWTs, applies role and current-plan permissions, validates serializer input and persists through PostgreSQL. Redis distributes realtime room events.

Deployment and PWA

- compose.yaml defines frontend Nginx, backend Daphne, PostgreSQL and Redis with health/dependency wiring and persistent volumes.
- The backend entrypoint waits for dependencies, applies migrations and collects static assets.
- The manifest, icons and registered service worker provide installability and offline shell startup.
- Authenticated API responses and media are deliberately excluded from service-worker caches to prevent stale or private-data reuse.

3. Domain model and server-enforced rules

Aggregate	Key relationships
Identity	User 1:1 UserPreference; verified artist 1:1 ArtistProfile; User 1:N ArtistApplication reviews; User M:N following.
Subscriptions	SubscriptionPlan 1:N UserSubscription and SubscriptionOrder; orders preserve quoted prices and gateway state.
Music	Artist(User) 1:N Album/Song; Album 1:N Song; Playlist M:N Song through PlaylistSong; Song stores collaborators/files.
Listening	User and Song create Stream/RecentlyPlayed rows; temporary ListeningGroup has members and current playback state.
Operations	Ticket/messages, preferences-driven notifications and monthly ArtistPayout rows link to users by role.

Access and integrity rules

- Playlist creation rejects requests after the active Free/Silver/Gold limit; this is not dependent on frontend checks.
- Stream creation checks the daily allowance before creating a Stream or incrementing Song.play_count.
- Early-access albums and songs are filtered from catalog, detail, search, chart, playlist serialization, history, download and realtime entry points unless entitled.
- Gold listeners, administrators and the owning artist may read per-song statistics; public artist profiles hide aggregate streams unless entitled.
- Distinct listeners derive from Stream.user IDs, and revenue derives from the configured per-stream rate rather than UI constants.
- Only verified artists publish; ownership protects mutation and upload endpoints; supported audio and image files are validated server-side.
- Support/admin roles manage tickets, while only administrators change plan prices and payout/audit state.

4. Implemented requirement coverage

Requirement group	Delivered implementation
Authentication	Listener and artist registration, login, JWT refresh/logout, privacy acknowledgement, password reset and artist review.
Discovery/profiles	Home, search, recent listening mapped to containing playlists, albums/singles, public/private profiles, follows and verification.
Playback	Play/pause, seek, queue, shuffle, repeat, lyrics, actual audio upload/playback, stream accounting and server limits.
Artist tools	Verified-artist album/song CRUD, persisted collaborators, image/audio uploads, metadata, lyrics and calculated statistics.
Plans/payment	Free/Silver/Gold limits/features, durations, admin prices, Zarinpal sandbox request/verification and entitlement activation.
Operations	Preferences-driven release/expiry notifications, support chat, API-backed audit UI and monthly reports with unique listeners.
Optional	Invite-code temporary listening groups, realtime synchronized state/progress and last-member cleanup.

Validation evidence (15 August 2026)

- Frontend production TypeScript/Vite build passed.
- Frontend suite passed: 33 files, 166 tests, 0 failures.
- Django system check passed with 0 issues; makemigrations --check reports no missing model changes.
- Docker Compose configuration resolves successfully.
- The complete backend database suite could not run locally because PostgreSQL was unavailable at localhost:5433; four database-independent tests passed before setup errors. This is an environment blocker, not reported as a green suite.
- Container startup, PostgreSQL/Redis integration, payment callback and WebSocket smoke tests still require a running Docker Desktop daemon and network access to the sandbox gateway.

5. AI-assisted development by phase

AI was used as an implementation/review aid. Suggestions were reviewed against the assignment and repository, then validated through compilation, tests, migration checks and authorization-focused inspection.

Phase 1 example - typed API boundary

AI helped identify that UI state and API payloads needed an explicit translation layer rather than leaking snake_case throughout components.

```
function mapSong(dto: SongResponse): Song {  
  return { id: dto.id, artistName: dto.artist_name,  
    coverImage: dto.cover_image ?? undefined };  
}
```

Phase 2 example - backend entitlement enforcement

AI review found playlist and stream limits enforced only or partly in presentation code. The backend mutation paths were hardened before persistence.

```
limit = get_playlist_limit(request.user)  
if limit is not None and Playlist.objects.filter(  
  created_by=request.user).count() >= limit:  
  raise DomainError('playlist_limit_reached', ..., status_code=403)
```

Phase 3 example - early-access leak prevention

AI-assisted tracing followed early-access songs through nested playlist serializers and removed an indirect public-data leak.

```
visible = Q(song__album__isnull=True) | Q(  
  song__album__is_early_access=False)  
return rows.filter(visible)
```

Human verification and limitations

- Generated changes were checked against model ownership, role semantics, serializer fields and UI types before acceptance.
- Focused tests cover subscription limits, statistics, distinct listeners, notifications and nested early-access visibility.
- No AI output is treated as proof of runtime behavior; external services and unavailable infrastructure are documented explicitly.

6. Final delivery checklist

- Apply Django migrations 0005_playlist_cover_image and payments 0002_artistpayout_unique_listeners.
- Run docker compose up --build on a host with Docker Desktop active and wait for PostgreSQL/Redis/backend/frontend health.
- Run the complete backend pytest suite in that environment and retain the log; rerun npm test -- --run and npm run build.
- Verify Zarinpal sandbox redirect/callback using deployment-specific merchant and callback environment values.
- Open two authenticated browsers and smoke-test invite joining, play/pause/song/progress synchronization and last-member cleanup.
- Confirm PWA installation and offline shell behavior, while ensuring authenticated API responses are absent from browser caches.
- Keep .env secrets, database/media test data, dependency directories, cache files and coverage artifacts out of Git.

Final status

Repository implementation covers the mandatory assignment areas and the optional synchronized group-listening feature. Frontend, static backend, migration and Compose checks are green. Full database/container/external-gateway runtime confirmation remains an explicit pre-submission environment step.